AIML DEVELOPER BUILD GUIDE

ROBIN OSINT

Complete Cursor Build Prompts

From empty folder to deployed system. 40 steps. 10 phases. 7 days.

40	10	42	23	7
Total Steps	Phases	Cursor Prompts	Manual Steps	Day Build

Internal Use Only — February 2026 — ROBIN OSINT v1.0

How to Use This Document

This document contains 40 steps organized into 10 phases. Execute them in exact order — each step depends on the previous one being complete and verified.

CURSOR PROMPT
Paste this exact text into Cursor AI chat. Do not paraphrase.

MANUAL STEP
Perform this action in a browser, terminal, or external tool.

VERIFY
Confirm this before proceeding. If it fails, debug before moving on.

> i Start with Step 01 and do not skip steps. Create .cursorrules (Step 40) on Day 1, not Day 7 — it makes every other prompt more accurate.

PHASE 01

Environment Setup

01 Install Required Software

Node.js, Git, Cursor IDE — your local environment

MANUAL STEP

WHAT TO DO

1. Go to nodejs.org → download Node.js v20 LTS → install. Verify: node --version must show v20.x.x
2. Go to git-scm.com → download Git for your OS → install with defaults. Verify: git --version
3. Go to cursor.com → download Cursor → install → sign in → Settings → Models → select Claude Sonnet or GPT-4o
4. Install Postman from postman.com — you will test every API endpoint here before the Full Stack Developer connects

> ✓ Terminal shows node v20+, git version exists, Cursor opens without errors.

02 Create GitHub Repositories

robin-backend (yours) and robin-frontend (Full Stack Developer)

MANUAL STEP

WHAT TO DO

5. Go to github.com → New Repository → robin-backend → Private → do NOT initialize with README → Create
6. Create second repo: robin-frontend → Private → do NOT initialize → Create
7. On your computer create folder: robin-osint → inside it two folders: robin-backend and robin-frontend
8. Terminal → cd robin-osint/robin-backend → git init → git remote add origin https://github.com/YOUR_USERNAME/robin-backend.git
9. When Full Stack Developer gives you their GitHub username: repo Settings → Collaborators → Add them to robin-frontend only

> ✓ Both repos exist on GitHub. robin-backend folder exists locally and is connected to GitHub remote.

03 Create Supabase Project

Your database, auth, and storage — the infrastructure backbone

MANUAL STEP

WHAT TO DO

10. Go to supabase.com → Sign up → Create New Project → name: robin-osint → choose region closest to your market (Singapore or Mumbai for Pakistan) → Create. Wait 2 minutes.
11. Go to Settings → API. Copy and save securely: Project URL, anon/public key, service_role key. Label them clearly.
12. Go to Settings → Database. Copy Connection String (URI) and the Connection Pooling string. Use the pooling string in production.
13. Go to Database → Extensions → search "vector" → click Enable. This enables pgvector for AI embeddings.
14. Go to Authentication → Providers → Google → enable it. Leave this tab open for Step 4.

✓ Supabase project shows Healthy status. pgvector shows as enabled. All 3 keys saved securely.

! CRITICAL: Never put the service_role key in any file committed to GitHub. It bypasses all Row Level Security.

04 Set Up Google OAuth

Google Cloud Console → OAuth credentials → connect to Supabase

MANUAL STEP

WHAT TO DO

15. Go to console.cloud.google.com → Create New Project → name: robin-osint
16. Left sidebar → APIs & Services → OAuth consent screen → External → fill App name: ROBIN OSINT, your email → Save
17. Go to APIs & Services → Credentials → Create Credentials → OAuth 2.0 Client ID → Web application → Name: ROBIN OSINT Web
18. In Authorized redirect URIs add: https://YOUR_SUPABASE_PROJECT_ID.supabase.co/auth/v1/callback
19. Click Create → copy Client ID and Client Secret → go back to Supabase → Authentication → Google provider → paste them → Save

✓ Supabase Authentication → Providers → Google shows as Enabled with your credentials saved.

05 Get Groq API Key

Your AI provider — free tier is sufficient for development

MANUAL STEP

WHAT TO DO

20. Go to console.groq.com → Sign up → verify email → log in

21. Go to API Keys → Create API Key → name: robin-osint-production → copy the key immediately (shows once only) → save securely
22. Model for analysis: llama-3.1-70b-versatile. Model for embeddings: nomic-embed-text. Both available on free tier.

> ✓ Groq API key saved. Test it in Groq playground by sending a simple message.

PHASE 02

Database Schema & Security

06 Open robin-backend in Cursor, Create Schema File

First Cursor prompt — generates the complete database schema

CURSOR PROMPT

BEFORE YOU PASTE THIS PROMPT

Open Cursor → File → Open Folder → select your robin-backend folder. Press Cmd+L (Mac) or Ctrl+L (Windows) to open the AI chat panel.

CURSOR PROMPT — PASTE EXACTLY AS WRITTEN

```
Create a file called database/schema.sql with a complete PostgreSQL schema
for a multi-tenant news intelligence SaaS called ROBIN OSINT.

-- All UUIDs use gen_random_uuid() as default
-- All timestamps use timezone-aware timestamptz with default now()
-- Add indexes on all foreign keys and frequently queried columns

Create these 12 tables in this exact order:

1. clients — companies subscribed to the platform
   Columns: id, name TEXT NOT NULL, industry TEXT, custom_prompt TEXT,
created_at

2. users — all people with access (linked to Supabase Auth)
   id UUID (references auth.users), email TEXT UNIQUE NOT NULL, full_name TEXT,
   role TEXT CHECK (role IN (SUPER_ADMIN,ADMIN,ANALYST)) NOT NULL,
   client_id UUID REFERENCES clients(id) ON DELETE CASCADE (nullable for
SUPER_ADMIN),
   created_at, last_seen_at

3. sources — news sources each company monitors
   id, client_id NOT NULL, name TEXT, url TEXT NOT NULL,
   source_type TEXT CHECK (source_type IN (rss,html,browser)) DEFAULT rss,
   is_active BOOLEAN DEFAULT true, last_scraped_at TIMESTAMPTZ,
last_scrape_error TEXT,
   scrape_success_count INT DEFAULT 0, scrape_fail_count INT DEFAULT 0,
created_at

4. watch_keywords — keywords each company monitors
   id, client_id NOT NULL, keyword TEXT NOT NULL, is_active BOOLEAN DEFAULT
true,
   created_at. Add UNIQUE constraint on (client_id, keyword)

5. articles — every scraped news article
   id, client_id NOT NULL, source_id REFERENCES sources(id),
   title TEXT, content TEXT, url TEXT NOT NULL (UNIQUE), content_hash TEXT
(UNIQUE),
   published_at TIMESTAMPTZ, matched_keywords TEXT[] DEFAULT {},
   is_tagged BOOLEAN DEFAULT false,
   analysis_status TEXT CHECK (status IN (pending,processing,complete,failed))
DEFAULT pending,
   embedding vector(768), created_at
```

```
6. article_analysis — AI-generated results (separate from articles)
   id, article_id UUID REFERENCES articles(id) ON DELETE CASCADE UNIQUE,
   summary TEXT, sentiment TEXT CHECK (sentiment IN
(positive,negative,neutral)),
   importance_score INTEGER CHECK (1 to 10),
   importance_reason TEXT,
   narrative_frame TEXT CHECK
(crisis,recovery,accountability,conflict,human_interest,economic,none),
   entities JSONB DEFAULT {"people":[],"orgs":[],"locations":[],"figures":[]},
   claims JSONB DEFAULT [], created_at

7. entity_mentions — normalized entity tracking for cross-article analysis
   id, article_id REFERENCES articles(id), client_id NOT NULL,
   entity_name TEXT, entity_type TEXT CHECK (person,org,location,figure),
created_at
   Index on (client_id, entity_name, entity_type)

8. narrative_patterns — output of nightly batch intelligence job
   id, client_id NOT NULL, pattern_date DATE NOT NULL,
   sentiment_breakdown JSONB, volume_data JSONB, top_entities JSONB,
   anomalies JSONB DEFAULT [], entity_cooccurrences JSONB DEFAULT [],
   source_sentiment JSONB DEFAULT [], weekly_narrative TEXT,
   risk_level TEXT CHECK (low,medium,elevated,high) DEFAULT low, created_at
   UNIQUE on (client_id, pattern_date)

9. article_files — media files attached to articles
10. reports — generated PDF intelligence reports
11. chat_history — analyst chat conversations
12. system_state — system-level flags (scraper lock)

After tables, add indexes on: articles(client_id), articles(published_at DESC),
articles(analysis_status), entity_mentions(client_id, entity_name)

Add pgvector index:
CREATE INDEX ON articles USING ivfflat (embedding vector_cosine_ops) WITH
(lists=100);
```

AFTER CURSOR GENERATES

23. Review the file: database/schema.sql — confirm all 12 tables are present
24. Go to Supabase → SQL Editor → New Query → paste the full schema.sql content → Run → wait for Success message
25. Go to Table Editor — confirm you can see all 12 tables listed

✓ All 12 tables visible in Supabase Table Editor. No SQL errors during execution.

! If you get "relation already exists" error: run DROP SCHEMA public CASCADE; CREATE SCHEMA public; then re-run. Only do this on a fresh project with no data.

07 Generate Row Level Security Policies

Database-level data isolation — most critical security feature

CURSOR PROMPT

CURSOR PROMPT

```
Create a file called database/rls-policies.sql with complete Row Level Security
policies for all ROBIN OSINT tables.

-- REQUIREMENTS:
-- 1. Enable RLS on every table that contains company data
-- 2. SUPER_ADMIN users bypass RLS (can see everything)
-- 3. All others can only see rows where client_id matches their JWT claim

The JWT custom claim for client_id is accessed via:
((auth.jwt() ->> user_metadata)::jsonb ->> client_id)::uuid

Role is accessed via:
(auth.jwt() ->> user_metadata)::jsonb ->> role

Generate:
1. ALTER TABLE statements to enable RLS on all 11 data tables
2. For each table create SELECT, INSERT, UPDATE, DELETE policies:
   SUPER_ADMIN sees/modifies all, others only their client_id
3. users table special rules:
   - Users can always see/update their own row (auth.uid() = id)
   - SUPER_ADMIN sees all users
   - ADMIN sees users in their own client_id
4. system_state table: deny all access from authenticated roles (backend only)
5. Add comments before each table block explaining what it protects

-- Run this in Supabase SQL Editor AFTER schema.sql
```

AFTER GENERATING

26. Go to Supabase SQL Editor → New Query → paste rls-policies.sql → Run
27. Go to Authentication → Policies — you should see entries for each table

✓ Supabase Authentication → Policies shows multiple policies for each table.

08 Create pgvector Similarity Search Function

Powers the AI chat feature — must exist before chat endpoint is built

CURSOR PROMPT

CURSOR PROMPT

```
Create database/functions.sql with PostgreSQL functions for ROBIN OSINT.

Function 1: match_articles
Parameters: query_embedding vector(768), p_client_id uuid,
            match_threshold float DEFAULT 0.7, match_count int DEFAULT 20
Returns: table of (id, title, content, url, published_at, source_id,
         sentiment, importance_score, summary, similarity float)
Logic: JOIN articles + article_analysis, filter by client_id,
       use cosine similarity: 1 - (embedding <=> query_embedding),
       WHERE similarity > threshold, ORDER BY similarity DESC, LIMIT match_count

Function 2: get_sentiment_trend
Parameters: p_client_id uuid, p_days int DEFAULT 30
```

```
Returns: (date, positive_count, negative_count, neutral_count, total) per day

Function 3: get_entity_cooccurrences
Parameters: p_client_id uuid, p_limit int DEFAULT 20
Returns: (entity_a, entity_b, co_count) — self-join on same article_id

-- Run in Supabase SQL Editor after rls-policies.sql
```

> ℹ Run in Supabase SQL Editor. These functions are called by API endpoints later.

PHASE 03

Backend Project Scaffolding

09 Initialize Node.js Project & Install All Dependencies

package.json, all packages, folder structure

CURSOR PROMPT

CURSOR PROMPT

```
Initialize a production-grade Node.js backend project for ROBIN OSINT.

1. Create package.json with:
   - type: "module" (ES modules — import/export throughout, never require)
   - scripts: start, dev (node --watch), scrape (node src/scrapers/run-once.js)

2. All production dependencies to install:
   express, @supabase/supabase-js, groq-sdk, crawlee,
   playwright, cheerio, rss-parser, @mozilla/readability,
   jsdom, node-cron, zod, cors, helmet,
   express-rate-limit, dotenv, uuid

3. Create this exact folder structure with placeholder files:
   src/index.js
   src/routes/ (articles, analytics, sources, keywords, users, clients, chat,
reports)
   src/middleware/ (auth, roleCheck, rateLimiter)
   src/scrapers/ (orchestrator, rss-crawler, html-crawler, browser-crawler,
run-once)
   src/services/ (keyword-matcher, dedup-checker, article-saver, embedding)
   src/ai/ (analysis-worker, batch-intelligence, chat-rag)
   src/scheduler/cron.js
   src/lib/ (supabase, groq)
   database/ (schema, rls-policies, functions already exist)
   .env.example  .gitignore  README.md

4. .gitignore: node_modules, .env, .env.local, dist, .DS_Store, *.log

5. .env.example with empty values for:
   SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY,
   GROQ_API_KEY, PORT, NODE_ENV, FRONTEND_URL, SCRAPER_INTERVAL_HOURS

// Do NOT run npm install — I will do that manually in terminal
```

AFTER CURSOR CREATES THE FILES

28. Open Cursor terminal: View → Terminal (or Ctrl+`)
29. Run: npm install — wait for all packages (2-3 minutes)
30. Run: npx playwright install chromium — installs browser for scraping
31. Create .env by copying .env.example and filling in all values from your saved credentials

✓ node_modules folder exists. All folders in src/ created. .env file created with real values.

10 Create Supabase & Groq Client Libraries

Shared instances used across the entire codebase

CURSOR PROMPT

CURSOR PROMPT

```
Build src/lib/supabase.js and src/lib/groq.js — singleton clients.

src/lib/supabase.js:
- Import createClient from @supabase/supabase-js
- Load SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY from process.env
- Throw descriptive error at startup if either env var is missing (fail fast)
- Create and export supabase client using SERVICE ROLE KEY (not anon key)
- Add comment: "Backend always uses service role. Never expose to frontend."

src/lib/groq.js:
- Import Groq from groq-sdk, load GROQ_API_KEY from env, throw if missing
- Export constants: ANALYSIS_MODEL = llama-3.1-70b-versatile
                    EMBEDDING_MODEL = nomic-embed-text
- Export helper: groqChat(messages, options)
  wraps groq.chat.completions.create with defaults:
  model: ANALYSIS_MODEL, temperature: 0.1, max_tokens: 1500
  includes try/catch with descriptive errors

// Use named exports throughout
```

✓ Both files exist, import env vars correctly, throw descriptive errors if vars are missing.

11 Create Express Server Entry Point

src/index.js — the server that ties everything together

CURSOR PROMPT

CURSOR PROMPT

```
Build src/index.js — the Express server entry point for ROBIN OSINT.

1. Apply middleware in this exact order:
   dotenv config (first), helmet(), cors() allowing only FRONTEND_URL,
   express.json() with 10mb limit, rate limiter from middleware/rateLimiter.js

2. Mount routes:
   /api/articles, /api/analytics, /api/sources, /api/keywords,
   /api/users, /api/clients, /api/chat, /api/reports

3. Add GET /health endpoint:
   Returns: { status: "ok", timestamp, service: "robin-backend" }
   Used by Railway to confirm service is running.

4. Add global error handler (4 params: err, req, res, next):
   Logs error with timestamp. Returns 500 with { error, message }.
   In production: omit message field.

5. Start server on PORT from env (default 3001)

6. Import and start cron scheduler (placeholder import)
```

```
7. Import and start AI analysis worker (placeholder import)

8. Handle SIGTERM for graceful shutdown (Railway sends this before stopping)

// If required env vars are missing: log clearly and process.exit(1)
```

TEST

32. Run: npm run dev — check it reaches "running on port 3001"
33. In Postman: GET http://localhost:3001/health → must return {"status":"ok"}

> ✓ GET /health returns 200 with status ok.

PHASE 04

Authentication & Authorization Middleware

12 Build JWT Auth Middleware

Every API route passes through this — must be bulletproof

CURSOR PROMPT

CURSOR PROMPT

```
Build src/middleware/auth.js — JWT verification for all API routes.

1. Extract Bearer token from Authorization header.
   If missing or wrong format: return 401 { error: "Missing authorization token"
}

2. Verify token using supabase.auth.getUser(token)
   If error or no user: return 401 { error: "Invalid or expired token" }

3. Fetch user profile from users table:
   SELECT id, email, full_name, role, client_id WHERE id = user.id
   If no record: return 403 { error: "User profile not found. Contact
administrator." }

4. Attach to req.user: { id, email, fullName, role, clientId }

5. Call next() to proceed.

6. Wrap all in try/catch. Unexpected errors → 500 { error: "Authentication
failed" }

Export: export const authenticate = async (req, res, next) => { ... }

---

Also build src/middleware/roleCheck.js:

requireRole(allowedRoles) — returns middleware that:
  - Checks req.user.role is in allowedRoles
  - If not: return 403 { error: "Insufficient permissions" }

requireSameClient() — returns middleware that:
  - SUPER_ADMIN: calls next() immediately
  - Others: verify clientId in params/body/query matches req.user.clientId

---

Also build src/middleware/rateLimiter.js using express-rate-limit:
  defaultLimiter: 100 requests per 15 minutes per IP
  authLimiter: 10 requests per 15 minutes
  chatLimiter: 20 requests per hour per IP
```

✓ Three middleware files created. auth.js imports supabase correctly.

> **!** authenticate middleware must run on EVERY route. One unprotected route can expose all client data.

PHASE 05

The Scraper System

13 Build Keyword Matcher Service

Determines if an article is relevant to a specific client

CURSOR PROMPT

CURSOR PROMPT

```
Build src/services/keyword-matcher.js

matchKeywords(text, keywords) → string[]
- Case-insensitive substring match, handles multi-word phrases
- Returns matched keywords array (deduplicated)
- Precompile regex patterns for performance

hasMinimumMatch(text, keywords, minMatches=1) → boolean
- Quick filter: returns true if at least minMatches keywords found

matchArticle({title, content}, keywords) → {matched, matchedKeywords}
- Searches both title and content
- Keyword in TITLE always includes (regardless of content)
- Returns matched boolean + which keywords matched
```

14 Build Deduplication Checker

Two-layer check — URL uniqueness and content fingerprint

CURSOR PROMPT

CURSOR PROMPT

```
Build src/services/dedup-checker.js

generateContentHash(content) → string
- Take first 600 chars of content
- Normalize: lowercase, remove extra whitespace and punctuation
- Return MD5 hex string using crypto (Node built-in)

isDuplicate(url, content, clientId) → Promise<{isDuplicate, reason}>
- Layer 1: Query articles WHERE url = url AND client_id = clientId
  → if found: return { isDuplicate: true, reason: "url_exists" }
- Layer 2: Generate hash, query WHERE content_hash = hash AND client_id
  → if found: return { isDuplicate: true, reason: "content_exists" }
- If neither: return { isDuplicate: false, reason: null }
- On DB error: log warning but return isDuplicate: false
  (better to store potential duplicate than miss real article)

normalizeUrl(url) → string
- Remove trailing slashes
- Remove UTM parameters (?utm_source, ?ref, etc.)
- Lowercase the domain portion
- Call this on every URL before dedup check
```

15 Build Article Saver Service

Validates, deduplicates, and persists articles to Supabase

CURSOR PROMPT

CURSOR PROMPT

```
Build src/services/article-saver.js

Define ArticleSchema with Zod:
- title: string min 5 max 500
- content: string min 100 chars
- url: string URL format
- publishedAt: coerce to Date
- sourceName, sourceId (UUID), clientId (UUID), matchedKeywords (array)

saveArticle(rawArticle) → Promise<{saved, articleId, reason}>
1. Validate with ArticleSchema → if invalid return { saved: false, reason:
"validation_failed" }
2. normalizeUrl()
3. isDuplicate() → if true return { saved: false, reason: "duplicate: " + reason
}
4. Generate content hash
5. Insert to articles table with analysis_status: "pending"
6. On success: log article title (truncated), return { saved: true, articleId }
7. On unique_violation (code 23505): return { saved: false, reason:
"duplicate_race_condition" }

updateSourceScrapeStatus(sourceId, success, errorMessage?)
- Updates: last_scraped_at = now()
- Increments scrape_success_count or scrape_fail_count
- Sets last_scrape_error if failed
```

16 Build RSS Crawler

Fast, reliable — handles ~60% of your news sources

CURSOR PROMPT

CURSOR PROMPT

```
Build src/scrapers/rss-crawler.js

fetchRssFeed(url) → Promise<RSSItem[]>
- RSSParser with timeout 15000ms, User-Agent: Mozilla/5.0
- Return items array, empty array on error (log, do not throw)

fetchFullArticleContent(url) → Promise<{title, content, publishedAt}>
- fetch() with 15s timeout and browser User-Agent header
- Parse with JSDOM, extract main content using Mozilla Readability
- Fallback to article selector or body text if Readability returns null
- Clean: remove excessive whitespace, trim to max 15000 chars

crawlRssSource(source, keywords, options?) → Promise<CrawlResult>
  CrawlResult: { sourceId, articlesFound, articlesSaved, errors[] }

Logic:
1. Fetch RSS feed from source.url
2. Filter: only items published AFTER source.last_scraped_at (or last 48hr if
null)
```

```
3. For each RSS item (max 50 per run):
   a. Quick keyword match on RSS title + description (fast, no HTTP request)
   b. If no match: skip
   c. If match: fetch full article content from item.link
   d. Re-run keyword match on full content
   e. If matches: call saveArticle()
4. Concurrency: process max 3 articles simultaneously via Promise.allSettled
batching
5. Return CrawlResult with stats
```

TEST THIS STEP

34. Add test block at bottom pointing to https://feeds.bbci.co.uk/news/world/rss.xml with keyword "economy"
35. Run: node src/scrapers/rss-crawler.js — check Supabase articles table for new rows
36. Remove the test block after confirming

✓ Running the test produces rows in Supabase articles table with analysis_status = "pending".

17 Build HTML Crawler (Cheerio)

For static sites without RSS — no browser needed

CURSOR PROMPT

CURSOR PROMPT

```
Build src/scrapers/html-crawler.js

fetchPage(url) → Promise<string|null>
- fetch() with 20s timeout, User-Agent: Mozilla/5.0 (Googlebot compatible)

extractArticleLinks(html, baseUrl) → string[]
- Load into Cheerio, find all <a> tags with href
- Filter: links containing /news/, /article/, /story/, /post/, date pattern
- Convert relative URLs to absolute, deduplicate, return max 30 links

crawlHtmlSource(source, keywords) → Promise<CrawlResult>
1. Fetch homepage HTML
2. Extract article links
3. For each link (max 20, parallel batches of 3):
   a. Fetch full page
   b. Extract content using Readability + JSDOM
   c. If keywords match: saveArticle()
4. Add exponential backoff: if request fails, wait 2s then retry once
5. Return CrawlResult stats
```

18 Build Browser Crawler (Playwright)

Heavy artillery for JS-heavy and anti-bot protected sources

CURSOR PROMPT

CURSOR PROMPT

```
Build src/scrapers/browser-crawler.js

launchBrowser() → Promise<Browser>
- headless: true
- args: [--no-sandbox, --disable-setuid-sandbox, --disable-dev-shm-usage,
--disable-gpu]
  (required for Railway/Docker environments)

MAX_CONCURRENT_PAGES = 2 (never open more than 2 tabs at once)

crawlBrowserSource(source, keywords) → Promise<CrawlResult>

CRITICAL: Always close browser in finally block even on error.

1. Launch browser
2. Set extra headers: { Accept-Language: en-US,en;q=0.9 }
3. Set viewport: { width: 1280, height: 800 }
4. Navigate to source.url, waitUntil: networkidle, timeout: 30000
5. page.waitForTimeout(2000)
6. Get page HTML via page.content()
7. Extract article links using Cheerio on rendered HTML
8. For each article (max 10):
   a. Open new tab
   b. Navigate, wait for: article, main, .article-body selector
   c. Extract content with Readability
   d. Match keywords, save if matched
   e. Close tab (page.close())
9. Close browser in finally block
10. Log memory every 5 articles: process.memoryUsage().heapUsed
```

! Browser crawling uses ~800MB RAM per instance. Max concurrency must stay at 2. If Railway crashes, reduce to 1.

19 Build the Scraper Orchestrator

Coordinates all crawlers across all clients

CURSOR PROMPT

CURSOR PROMPT

```
Build src/scrapers/orchestrator.js

SCRAPER_LOCK_KEY = "scraper_running"

acquireLock() → Promise<boolean>
- Check system_state table WHERE key = SCRAPER_LOCK_KEY
- If locked and updated_at < 2 hours ago: override (stuck run), log warning
- If locked and recent: return false
- Upsert { key, value: "true", updated_at: now() }
- Return true

releaseLock() → Promise<void>
- Upsert { key, value: "false", updated_at: now() }

fetchAllActiveSources() → Promise<SourceWithKeywords[]>
```

```
- Query sources WHERE is_active = true
- For each source, fetch watch_keywords for that source client_id

runScraperCycle() → Promise<void>
1. Acquire lock → if fails: log "Scraper already running" and return
2. Log: === Scraper cycle started at [timestamp] ===
3. Wrap in try/finally (release lock in finally)
4. Fetch all active sources
5. Process RSS sources: parallel, concurrency 5
6. Process HTML sources: parallel, concurrency 3
7. Process Browser sources: SEQUENTIAL (memory intensive)
8. Log final summary: articles found, saved, errors per source
9. Log: === Scraper cycle completed in [duration]ms ===

Also export: scrapeSourceById(sourceId) for manual API trigger
```

TEST

37. Create src/scrapers/run-once.js: import { runScraperCycle } from ./orchestrator.js; runScraperCycle().then(() => process.exit(0))
38. Add a test source in Supabase Table Editor → sources
39. Run: node src/scrapers/run-once.js — articles should appear in Supabase

> ✓ run-once.js completes without errors. Articles appear in Supabase with analysis_status = "pending".

20 Build Cron Scheduler

The timer that automatically triggers the scraper

CURSOR PROMPT

CURSOR PROMPT

```
Build src/scheduler/cron.js

- Read SCRAPER_INTERVAL_HOURS from env (default: 2)
- Convert to cron: 2hrs → "0 */2 * * *", 4hrs → "0 */4 * * *"

startScheduler() → void
- Log: "Cron scheduler starting. Scraper runs every X hours."
- Schedule scraper: cron.schedule(cronExpression, async () => {
    try { await runScraperCycle(); }
    catch(err) { console.error("[Cron] Failed:", err.message); }
  })
- Schedule nightly batch: "0 2 * * *" → runBatchIntelligence()
- Log both schedules clearly

triggerScrapeNow() → Promise<void>
- Calls runScraperCycle() immediately
- Used by the manual trigger API endpoint
```

PHASE 06

AI Analysis Pipeline

21 Build Embedding Service

Converts article text to vectors — powers semantic search and chat

CURSOR PROMPT

CURSOR PROMPT

```
Build src/services/embedding.js

generateEmbedding(text) → Promise<number[]>
- Prepare: title + first 1200 chars of content, clean whitespace
- Call Groq embeddings API with EMBEDDING_MODEL
- Return response.data[0].embedding
- On error: log warning, return null (embedding failure should NOT block article
save)
- Add retry: if call fails, wait 2s and try once more

generateAndStoreEmbedding(articleId) → Promise<boolean>
- Fetch article title + content from Supabase
- Generate embedding
- If null: log warning, return false
- UPDATE articles SET embedding = embedding WHERE id = articleId
- Return true on success

generateEmbeddingForQuery(query) → Promise<number[]|null>
- Same as generateEmbedding but for a search query string
- Used by the chat RAG system
```

22 Build the AI Analysis Worker

Core AI pipeline — runs continuously, analyzes every article

CURSOR PROMPT

CURSOR PROMPT

```
Build src/ai/analysis-worker.js

ANALYSIS_PROMPT (template function taking article + clientName + industry +
keywords):

System: "You are an expert intelligence analyst specializing in media monitoring
and corporate communications risk assessment. Return only valid JSON."

User prompt:
"Analyze this article for [clientName] ([industry]).
Matched keywords: [keywords].
Title: [title]
Content: [first 3000 chars]
Published: [date] | Source: [source_name]

Return ONLY a JSON object with:
{
```

```
  summary: "3 specific factual sentences",
  sentiment: "positive|negative|neutral",
  importance_score: 1-10,
  importance_reason: "one sentence re: impact on [clientName]",
  narrative_frame:
"crisis|recovery|accountability|conflict|human_interest|economic|none",
  entities: { people: [], orgs: [], locations: [], figures: [] },
  claims: [{ claim, source, verified: bool }]
}

Scoring: 9-10=regulatory/legal/scandal, 7-8=financial/leadership,
5-6=industry trends, 3-4=general mention, 1-2=tangential"

analyzeArticle(article) → Promise<boolean>
1. UPDATE articles SET analysis_status="processing"
2. Fetch client name + industry
3. Build prompt and call groqChat()
4. Strip markdown fences, JSON.parse() in try/catch
5. Save to article_analysis table
6. Save entities to entity_mentions table (one row per entity)
7. generateAndStoreEmbedding(article.id)
8. UPDATE articles SET analysis_status="complete"
On any error: UPDATE analysis_status="failed", log error with articleId

startAnalysisWorker() → void
- Poll every 30 seconds for articles WHERE analysis_status="pending"
- Fetch 5 at a time, process sequentially with 500ms delay between each
- Never throw — always continue the loop on errors
- Log every 10th analysis: sentiment + score
```

TEST

40. If you have pending articles from the scraper test, run the worker and watch logs
41. Check article_analysis table — rows should appear within 30 seconds

✓ article_analysis table has rows. Articles show analysis_status = "complete". All JSON fields populated.

! If Groq returns non-JSON text, parsing fails. Add console.log of raw Groq response during testing.

23 Build Nightly Batch Intelligence Job

Cross-article pattern detection — runs at 2am every night

CURSOR PROMPT

CURSOR PROMPT

```
Build src/ai/batch-intelligence.js — nightly cross-article analysis.

runBatchIntelligence() → Promise<void>
For each active client:

Step A — Collect raw data (last 30 days):
```

```
- sentiment_breakdown: { positive, negative, neutral, total, percentages }
- volume by day: [{ date, count }]
- this_week vs last_week sentiment comparison
- top entity mentions: GROUP BY entity_name COUNT DESC LIMIT 20
- volume anomalies: flag if today > 3x the 7-day average
- entity co-occurrences: use get_entity_cooccurrences DB function
- per-source sentiment: negative% per source

Step B — Detect anomalies:
Build anomalies array with objects:
{ type, severity: "high|medium", description, detected_at }
Check: volume_spike, sentiment_shift (>20pt swing vs last week),
       new_entity (first time appearing), source_tone_change

Step C — Generate narrative using Groq:
Send stats to Groq:
"Based on this 30-day data for [clientName]: [stats]
Provide:
1. The most significant narrative shift and its likely cause (2-3 sentences)
2. The single most actionable recommendation for comms team (1-2 sentences)
3. risk_level: low | medium | elevated | high
Return JSON: { narrative_shift, recommendation, risk_level }"

Step D — Save to narrative_patterns table (upsert on client_id +
pattern_date=today)

Log: "[Batch] Client [name]: risk=[level], anomalies=[count]"
```

24 Build the RAG Chat Engine

The AI chat feature — most complex, most valuable

CURSOR PROMPT

CURSOR PROMPT

```
Build src/ai/chat-rag.js — RAG engine for analyst chat.

retrieveRelevantArticles(question, clientId) → Promise<Article[]>
1. Generate embedding for the question
2. Call supabase.rpc("match_articles", { query_embedding, p_client_id,
   match_threshold: 0.65, match_count: 20 })
3. Also run keyword fallback: search articles.matched_keywords @> ARRAY[key
terms]
4. Merge + deduplicate, sort by similarity then recency
5. Return top 15 articles

getClientContext(clientId) → Promise<string>
- Fetch latest narrative_patterns row for this client
- Return formatted string with weekly_narrative + risk_level + anomalies summary

buildChatPrompt(question, articles, clientContext, clientName) → messages[]
System: "You are an intelligence analyst for [clientName]. Answer using ONLY
the provided article data. Cite dates and sources. If data does not support
a conclusion, say so. Never speculate beyond the evidence."

User: clientContext + relevant articles (formatted) + question

generateChatResponseStream(question, clientId, clientName) → AsyncGenerator
1. retrieveRelevantArticles()
```

```
2. getClientContext()
3. buildChatPrompt()
4. Call Groq with stream: true
5. yield each token: chunk.choices[0]?.delta?.content || ""
6. After stream ends: save full answer to chat_history table
```

PHASE 07

API Routes

25 Build Articles API Routes

The most-used endpoints — the analyst data feed

CURSOR PROMPT

CURSOR PROMPT

```
Build src/routes/articles.js — apply authenticate to ALL routes.

GET / — paginated article feed
Query params: page, limit (max 100), sentiment, source_id, keyword,
              from_date, to_date, min_score, tagged_only
All queries filter by req.user.clientId.
JOIN articles + article_analysis (left join) + sources
Order by published_at DESC
Return: { data: Article[], pagination: { total, page, limit, total_pages } }
Each article includes: id, title, url, published_at, source_name,
  matched_keywords, is_tagged, analysis_status,
  analysis: { summary, sentiment, importance_score, importance_reason,
              narrative_frame, entities } | null

GET /:id — single article detail
Return full article + analysis. 403 if client_id mismatch.

PATCH /:id/tag — toggle tagged status
Body: { is_tagged: boolean }
Roles: ADMIN, ANALYST

POST /trigger-scrape/:sourceId — manually trigger scrape
Roles: ADMIN, SUPER_ADMIN only
Verify source belongs to req.user.clientId
Call scrapeSourceById(sourceId)
```

26 Build Analytics API Routes

Charts data, sentiment trends, patterns

CURSOR PROMPT

CURSOR PROMPT

```
Build src/routes/analytics.js — all queries filter by req.user.clientId.

GET /sentiment
Returns: { breakdown: { positive, negative, neutral, total, percentages },
           trend: [{ date, positive, negative, neutral, total }],
           comparison: { this_week_positive_pct, last_week_positive_pct, change }
}

GET /volume
Returns: { daily: [{date, count}], total, average_per_day, peak_day, today }

GET /keywords
```

```
Returns per keyword: [{ keyword, total_articles, this_week, last_week, trend,
last_article_at }]

GET /sources
Returns per source: [{ source_id, source_name, total_articles, sentiment pcts,
                       last_scraped_at, success/fail counts,
avg_importance_score }]

GET /patterns
Return latest narrative_patterns row for this client.
If none exists: { message: "Intelligence patterns not yet generated." }

GET /summary — dashboard header stats
Returns: { articles_today, articles_this_week, high_importance_unread,
           negative_pct_today, risk_level }
```

27 Build Sources, Keywords, Users & Clients Routes

Management endpoints — all CRUD operations

CURSOR PROMPT

CURSOR PROMPT

```
Build these four route files. All require authenticate. All use Zod validation.

src/routes/sources.js:
  GET / — list sources for this client
  POST / — add source (ADMIN only) — validate: name, url, source_type
           check for duplicate URL before inserting
  DELETE /:id — soft delete (set is_active=false, preserve article history)
  PATCH /:id — update name, url, is_active, source_type

src/routes/keywords.js:
  GET / — list keywords for this client
  POST / — add keyword (ADMIN only) — normalize: trim + lowercase
           handle unique constraint violation gracefully
  DELETE /:id — delete (ADMIN only, verify ownership)

src/routes/users.js:
  GET / — list users for this client (ADMIN/SUPER_ADMIN)
  POST /invite — invite user (ADMIN only)
    supabase.auth.admin.inviteUserByEmail(email, { data: { client_id, role } })
    return: { message: "Invitation sent", email }
  DELETE /:id — remove user (ADMIN only)
  PATCH /:id/role — change role (ADMIN only, cannot change own role)
  GET /me — return req.user profile (any authenticated user)

src/routes/clients.js:
  GET / — list all clients (SUPER_ADMIN only)
  POST / — create client (SUPER_ADMIN only) — body: { name, industry }
  GET /:id — client details with user count + article count
  PATCH /:id — update name, industry, custom_prompt (SUPER_ADMIN only)
```

28 Build the Chat API Route

Streaming SSE endpoint — the feature that defines the product

CURSOR PROMPT

CURSOR PROMPT

```
Build src/routes/chat.js — SSE streaming chat endpoint.
Apply: authenticate, chatLimiter to all routes.

POST / — submit question, receive streaming AI response
Zod validate: { question: string min 5 max 1000 }

SSE streaming setup:
  res.setHeader("Content-Type", "text/event-stream")
  res.setHeader("Cache-Control", "no-cache")
  res.setHeader("Connection", "keep-alive")
  res.flushHeaders()

Logic:
1. Fetch client name for this req.user.clientId
2. Get stream from generateChatResponseStream(question, clientId, clientName)
3. For each token:
   res.write(`data: ${JSON.stringify({ token })}`)
4. When done: res.write("data: [DONE]") → res.end()
5. On client disconnect: req.on("close", abort)
6. On error: res.write error message then res.end()

GET /history — past chats for req.user.id
  Return last 20 chat_history rows ordered by created_at DESC
  Each: { id, question, answer (truncated 200 chars), created_at }

GET /history/:id — full chat entry (verify user_id = req.user.id)
```

TEST THE CHAT ENDPOINT

42. In Postman: POST to /api/chat with auth header and body {"question":"What articles do we have?"}
43. Use Postman "Send and Download" to see the SSE stream with data: {"token":"..."} events

✓ POST /api/chat returns streaming text/event-stream with data: {"token":"..."} events.

29 Build Reports Route & Connect All Routes

Final route + wire everything into src/index.js

CURSOR PROMPT

CURSOR PROMPT

```
Build src/routes/reports.js:

GET / — list reports for this client

POST /generate — generate intelligence report (ADMIN only)
Body: { title, date_from, date_to }
- Fetch articles + analysis for date range
- Fetch narrative_patterns for period
- Call Groq: "Generate an intelligence brief for [clientName]
  covering [date_from] to [date_to]. Include:
  executive summary, key themes, top 5 articles with summaries,
  sentiment analysis, risk assessment, recommended actions.
```

```
  Data: [stats + top articles]. Format as structured report."
- Save to reports table (file_url: null for MVP)
- Return report content as text + report ID

---

Then update src/index.js:
- Replace all placeholder imports with real route imports
- Add GET /api/admin/system-health (SUPER_ADMIN only)
- Run npm run dev — confirm server starts with no import errors

Test in Postman:
  GET /api/articles, GET /api/analytics/summary,
  GET /api/sources, GET /api/keywords, GET /health
```

✓ Server starts without errors. All routes respond. GET /health returns 200.

PHASE 08

Postman Collection & Test Data

30 Generate Postman Collection File

Hand this to the Full Stack Developer as his API reference

CURSOR PROMPT

CURSOR PROMPT

```
Create docs/postman-collection.json — valid Postman Collection v2.1.

Collection variables: baseUrl = http://localhost:3001, bearerToken = ""
Authorization pre-set at collection level: Bearer {{bearerToken}}

Organize into folders with all endpoints:
- Health: GET /health
- Articles: GET /, GET /:id, PATCH /:id/tag, POST /trigger-scrape/:sourceId
- Analytics: GET /sentiment, /volume, /keywords, /sources, /patterns, /summary
- Sources: GET /, POST /, PATCH /:id, DELETE /:id
- Keywords: GET /, POST /, DELETE /:id
- Users: GET /, POST /invite, GET /me, PATCH /:id/role, DELETE /:id
- Clients: GET /, POST /, GET /:id, PATCH /:id
- Chat: POST /, GET /history, GET /history/:id
- Reports: GET /, POST /generate

Each request: example request body, description, example response shape
```

AFTER GENERATING

44. Open Postman → Import → upload docs/postman-collection.json
45. Get a test JWT from Supabase Auth → paste into bearerToken collection variable
46. Test every endpoint. All should return 200 or meaningful 4xx responses.
47. Send the postman-collection.json file to the Full Stack Developer

✓ All endpoints tested. Collection sent to Full Stack Developer.

31 Create Seed Data Script

Populates DB with test data for development and demos

CURSOR PROMPT

CURSOR PROMPT

```
Create database/seed.js — Node.js script to populate test data.

Using supabase service role client, insert:

1. Two test clients:
   - National Bank of Pakistan (industry: Banking & Finance)
```

```
   - Pakistan Telecom Corp (industry: Telecommunications)

2. For National Bank, 5 RSS news sources:
   - Dawn Business: https://www.dawn.com/feeds/business
   - The News International: https://www.thenews.com.pk/rss/1/8
   - Business Recorder: https://www.brecorder.com/feeds
   - Reuters Pakistan: https://feeds.reuters.com/reuters/PKTopNews
   - Tribune Business: https://tribune.com.pk/feed

3. For National Bank, 8 watch keywords:
   "National Bank", "NBP", "banking sector", "SBP",
   "interest rate", "IMF Pakistan", "FATF", "financial fraud"

4. Log all inserted IDs at the end.

5. Create Super Admin user record:
   INSERT INTO users (id, email, full_name, role, client_id)
   VALUES (YOUR_AUTH_UUID, your@email.com, Your Name, SUPER_ADMIN, null)
   Note: auth user must already exist (log in via Google first)

// Run with: node database/seed.js
```

RUN THE SEED SCRIPT

48. First log in to your app once via Google OAuth to create your auth user. Get your UUID from Supabase → Authentication → Users.
49. Update the seed script with your UUID and email, then run: node database/seed.js
50. Verify in Supabase: clients has 2 rows, sources has 5 rows, watch_keywords has 8 rows

> ✓ Seed data visible in Supabase. Running the scraper now will produce real articles.

PHASE 09

Deployment to Railway

32 Prepare for Production Deployment

railway.toml, graceful shutdown, config centralization

CURSOR PROMPT

CURSOR PROMPT

```
Prepare robin-backend for production deployment on Railway.

1. Create railway.toml:
   [build] builder = "NIXPACKS"
   [deploy] startCommand = "npm start"
           healthcheckPath = "/health"
           healthcheckTimeout = 30
           restartPolicyType = "ON_FAILURE"
           restartPolicyMaxRetries = 3

2. Create .nvmrc with content: 20

3. Add to package.json: "engines": { "node": ">=20.0.0" }

4. Create src/config.js — centralize all env vars with validation:
   Throw descriptive errors if SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY,
   or GROQ_API_KEY are missing.
   Export typed config: { supabaseUrl, supabaseServiceKey, supabaseAnonKey,
   groqApiKey, port, nodeEnv, frontendUrl, scraperIntervalHours }

5. Update src/lib/ files to import from config.js instead of process.env
directly

6. Add graceful shutdown to src/index.js:
   process.on("SIGTERM", () => {
     console.log("Shutting down gracefully...");
     server.close(() => { process.exit(0); });
   })
   Allow 10 seconds for in-progress requests to complete.

7. Confirm .env is in .gitignore — run: git status and verify .env not listed
```

✓ Run npm run dev — confirm everything still works locally before pushing.

33 Push to GitHub

Final commit before deploying

MANUAL STEP

WHAT TO DO

51. In Cursor terminal: git add .

52. Run: git status — CONFIRM .env is NOT in the list. If it appears: stop, add .env to .gitignore first.
53. git commit -m "feat: initial ROBIN OSINT backend — scraper, AI pipeline, all API routes"
54. git push origin main

> ! CRITICAL: If you accidentally push .env or API keys, immediately rotate them in Supabase and Groq — even if the repo is private. Leaked keys are compromised keys.

34 Deploy to Railway

Connect GitHub → configure env vars → go live

MANUAL STEP

WHAT TO DO

55. Go to railway.app → Sign up with GitHub → New Project → Deploy from GitHub → select robin-backend
56. Railway detects Node.js and starts building. Wait 3-5 minutes.
57. Go to your service → Variables tab → add all env vars: SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY, GROQ_API_KEY, NODE_ENV=production, FRONTEND_URL (get from Full Stack Developer on Day 7), SCRAPER_INTERVAL_HOURS=2
58. After adding env vars Railway auto-redeploys. Wait for "Active" status.
59. Settings → Networking → Generate Domain → copy your Railway URL
60. Test: browser → https://YOUR_RAILWAY_URL/health → must return {"status":"ok"}
61. Send the Railway URL to the Full Stack Developer. He sets it as NEXT_PUBLIC_API_URL in Vercel.

> ✓ Railway shows Active status. /health returns 200 from public Railway URL. Logs show scraper cron and analysis worker started.

35 Update CORS for Production

Allow the Vercel frontend URL once Full Stack Developer deploys

CURSOR PROMPT

CURSOR PROMPT

```
Update src/index.js CORS to support multiple allowed origins:

const allowedOrigins = [
  process.env.FRONTEND_URL,    // production Vercel URL
  "http://localhost:3000",      // Full Stack Developer local dev
  "http://localhost:3001",      // local testing
].filter(Boolean);

cors({
  origin: (origin, callback) => {
```

```
    if (!origin) return callback(null, true); // allow Postman
    if (allowedOrigins.includes(origin)) return callback(null, true);
    callback(new Error(`CORS blocked: ${origin}`));
  },
  credentials: true,
  methods: ["GET", "POST", "PATCH", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"]
})

// After Full Stack Developer deploys to Vercel:
// Update FRONTEND_URL in Railway env vars to his production URL.
// Railway auto-redeploys — no code change needed.
```

PHASE 10

Final Testing & Maintenance

36 Final End-to-End Integration Test

Day 7 — verify the complete system before calling MVP done

MANUAL STEP

7 TESTS TO RUN IN ORDER

62. AUTH TEST: Open Full Stack Developer Vercel URL → click Login with Google → confirm you reach the Super Admin dashboard → verify your role shows correctly
63. SCRAPER TEST: In Railway logs confirm cron scheduler is running. Manually trigger POST /api/articles/trigger-scrape/[source_id] from Postman → confirm new articles appear in Supabase
64. AI PIPELINE TEST: Check article_analysis table — articles from step 2 should have analysis_status="complete" within 60 seconds. Confirm summary, sentiment, importance_score are all populated
65. DASHBOARD TEST: Confirm articles appear in the analyst dashboard via Full Stack Developer app. Sentiment charts show data. Article cards show AI summaries.
66. CHAT TEST: Open chat UI → ask "What are the most important articles from this week?" → confirm streaming response appears with real data from the database
67. RLS SECURITY TEST: Create a second test user with a different client_id. Log in as that user → confirm they see ZERO articles from the first client. This is the most critical test.
68. MEMORY TEST: Check Railway metrics — memory should be under 400MB with scraper idle. If over 600MB, check for browser crawler memory leaks.

✓ All 7 tests pass. MVP is complete and ready to demo.

! Do NOT skip the RLS security test (step 6). Multi-tenant data isolation is the single most critical security guarantee. If it fails, do not ship.

37 Create System Health Endpoint

Morning check — know when things break before clients notice

CURSOR PROMPT

CURSOR PROMPT

```
Add GET /api/admin/system-health endpoint (SUPER_ADMIN only).

Returns comprehensive health report:
{
  status: "healthy" | "degraded" | "critical",
  checked_at: ISO timestamp,
  scraper: {
    last_run: timestamp from system_state,
```

```
    is_locked: boolean,
    sources_total: n,
    sources_failing: n,  // fail_count > success_count in last 24hr
    articles_last_24h: n
  },
  ai_pipeline: {
    pending_articles: n,   // stuck pending for more than 1 hour
    failed_articles_24h: n,
    avg_analysis_time_minutes: n,
    completion_rate_pct: n
  },
  database: {
    total_articles: n,
    total_clients: n
  },
  costs: {
    articles_analyzed_this_month: n,
    estimated_groq_cost_usd: n  // rough: articles * 0.0005
  }
}

status logic:
  critical: pending_articles > 100 OR sources_failing > 50%
  degraded: pending_articles > 20 OR completion_rate < 90%
  healthy: everything else
```

38 Set Up JWT Custom Claims Hook

Required for role + client_id to appear in JWT tokens

CURSOR PROMPT

CURSOR PROMPT

```
Create database/jwt-hook.sql — Supabase custom access token hook.

Create a PostgreSQL function that:
1. Reads the user_id from the event parameter
2. Queries the users table: SELECT role, client_id WHERE id = user_id
3. Adds these to event.claims.user_metadata:
   { role: user.role, client_id: user.client_id }
4. Returns the modified event

The function signature for Supabase hook:
CREATE OR REPLACE FUNCTION public.custom_access_token_hook(event jsonb)
RETURNS jsonb LANGUAGE plpgsql AS $$
DECLARE
  claims jsonb;
  user_role text;
  user_client_id uuid;
BEGIN
  SELECT role, client_id INTO user_role, user_client_id
  FROM public.users WHERE id = (event->>user_id)::uuid;
  -- Add to claims and return modified event
END; $$;

After running in Supabase SQL Editor:
Go to Authentication → Hooks → Add hook:
Hook type: Custom Access Token
Function: public.custom_access_token_hook
```

69. Run jwt-hook.sql in Supabase SQL Editor
70. Go to Supabase → Authentication → Hooks → Add Custom Access Token hook → select the function
71. Test: log out and log back in → your new JWT should include role and client_id in user_metadata

> ✓ After logging in again, the auth middleware can now extract role and client_id from the JWT.

39 Add Error Monitoring

Know about errors before clients report them

CURSOR PROMPT

CURSOR PROMPT

```
Add structured error logging throughout the codebase.

Create src/lib/logger.js:
- Export log object with methods: info, warn, error, debug
- Each method: console.log with timestamp, level, and structured data
- In production (NODE_ENV=production): format as JSON for Railway log parsing
- In development: format as human-readable with colors

Format:
  [2026-02-23T09:00:00Z] [INFO] [Scraper] Cycle completed | articles=45
duration=12340ms
  [2026-02-23T09:00:01Z] [ERROR] [AI] Analysis failed | articleId=xxx error=...

Update these files to use logger instead of console.log:
- src/scrapers/orchestrator.js
- src/ai/analysis-worker.js
- src/ai/batch-intelligence.js
- src/middleware/auth.js
- src/index.js (startup logs)

Add to GET /api/admin/system-health:
- last_error: most recent ERROR log entry from the system
```

40 Create .cursorrules File

Teaches Cursor your architecture — every future prompt is more accurate

CURSOR PROMPT

CURSOR PROMPT — CREATE THIS ON DAY 1, NOT DAY 7

```
Create a file called .cursorrules in the project root with this content.
This file is read by Cursor automatically before every prompt.

# ROBIN OSINT Backend — Cursor Rules

## Project Context
```

```
Multi-tenant news intelligence SaaS.
Node.js backend with Express, Supabase, and Groq AI.

## Non-Negotiable Rules
1. Every database query MUST filter by client_id (unless SUPER_ADMIN)
2. Every route MUST have authenticate middleware applied
3. Never use Supabase anon key in backend code — always service role key
4. Always use ES modules (import/export) — never require()
5. Never expose service role key in any log output
6. All user input must be validated with Zod before processing
7. Wrap every async route handler in try/catch

## Code Style
- async/await only — never .then() chains
- Descriptive console.log with prefix: [Scraper], [AI], [Chat], [Auth]
- Route handlers thin — business logic in services/ or ai/ files
- Named exports preferred (except Express routers use default)

## Database Patterns
- Always use supabase from src/lib/supabase.js (singleton)
- const { data, error } = await supabase.from(...) — always check error
- Use .limit() on all queries — prevent accidental full-table scans

## Scraper Rules
- Never scrape more than 50 articles per source per run
- Always close Playwright browsers in finally blocks
- Never run more than 2 Playwright browsers simultaneously
- Always update last_scraped_at after each attempt (success or fail)
```

✓ .cursorrules exists in project root. From this point every Cursor prompt will automatically follow these rules — no need to re-explain architecture each time.

Build Summary

Phase	Deliverable	Day	Steps
01 — Environment Setup	Node.js, GitHub, Supabase, Groq accounts	Day 0	1–5
02 — Database Schema	12 tables, RLS policies, pgvector functions	Day 1	6–8
03 — Backend Scaffolding	Express server, all folders, Supabase + Groq clients	Day 1	9–11
04 — Auth Middleware	JWT verification, role checks, rate limiting	Day 1	12
05 — Scraper System	RSS, HTML, Browser crawlers + orchestrator + cron	Days 2–3	13–20
06 — AI Pipeline	Embeddings, analysis worker, batch job, RAG chat	Days 3–4	21–24
07 — API Routes	All 20 endpoints — articles, analytics, chat, management	Days 4–5	25–29
08 — Test Data	Postman collection, seed data, first admin user	Day 5	30–31
09 — Deployment	Railway deployment, env vars, CORS, GitHub push	Day 6	32–35
10 — Testing & Maintenance	End-to-end tests, health endpoint, logging, .cursorrules	Day 7	36–40

> **i** Execute steps in exact order. Verify each step before proceeding. If a verification fails, debug that step before moving to the next one.